

Sprint 1 Bootstrap: 36-State Factored MDP with LLM-Estimated Transitions

William Dennis

July 2026

1 MDP Formulation

The Sprint 1 simulator implements a finite-horizon factored MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma, T)$ with 36 states, 4 actions, and transitions estimated by an LLM (deepseek-v4-flash) using Algorithm 2.

1.1 State Space

Four factors: step bin (`step_bin` $\in \{\text{inactive}, \text{moderate}, \text{active}\}$), sleep quality (`sleep` $\in \{\text{good}, \text{poor}\}$), day type (`day_of_week` $\in \{\text{weekday}, \text{weekend}\}$), and burden level (`burden` $\in \{\text{low}, \text{medium}, \text{high}\}$):

$$\mathcal{S} = \{\text{inactive}, \text{moderate}, \text{active}\} \times \{\text{good}, \text{poor}\} \times \{\text{weekday}, \text{weekend}\} \times \{\text{low}, \text{medium}, \text{high}\}$$

Three factors are deterministic: `day_of_week` cycles daily through a 7-day pattern; `burden` uses a rolling window of the last 3 actions (increments on non-idle actions). Two factors are stochastic: `step_bin` at each timestep, and `sleep` at day boundaries.

1.2 Action Space

$$\mathcal{A} = \{\text{idle}, \text{movement_suggestion}, \text{goal_reminder}, \text{journal}\}$$

All non-idle actions incur a penalty of 0.05.

1.3 Transition Function

Transitions are loaded from 6 JSON probability tables generated by an LLM (deepseek-v4-flash, $N = 16$ samples per state-action pair, Algorithm 2):

- `day_boundary.json`: $P(\text{sleep}' \mid \text{step_bin}, \text{burden}, \text{day_of_week}, \text{sleep})$ – action-independent, evaluated at step 0 of each day.
- `within_day_0.json` through `within_day_4.json`: $P(\text{step_bin}' \mid \text{step_bin}, \text{burden}, \text{action}, \text{day_of_week}, \text{sleep})$ – one table per timestep, action-conditioned.

The LLM’s estimates reflect plausible real-world dynamics: interventions have modest effects on step counts, and sleep quality influences next-day activity.

1.4 Reward

$$R = \alpha \cdot \text{step_bin_value} + (1 - \alpha) \cdot \text{sleep_value} - \text{action_penalty}$$

with $\alpha = 0.9$. Step bin values: `inactive` $\rightarrow$ 0.0, `moderate` $\rightarrow$ 0.5, `active` $\rightarrow$ 1.0. Sleep values: `good` $\rightarrow$ 1.0, `poor` $\rightarrow$ -1.0. Action penalty: 0.0 for idle, 0.05 otherwise.

Optional `reward_multiplier_by_step` masks the last step of each day: [1, 1, 1, 1, 0].

1.5 Episode

$T = 90 \text{ days} \times 5 \text{ steps/day} = 450 \text{ timesteps per episode}$.

2 Upper and Lower Bounds

To calibrate agent performance we compute both simulation-based and exact bounds:

- **Optimal DP bound** via backward induction on the 384-state MDP (3 step bins $\times$ 2 sleep $\times$ 4^3 action histories for burden). This is the exact finite-horizon optimal value – any learning agent’s performance is bounded above by this.
- **Simulated reference policies** (10 seeds, 450 timesteps) for comparison.

Policy	Total Reward	Per Step	Description
Optimal DP (backward induction)	379.1	0.842	Exact upper bound, 384 states
Fixed idle	380.4 ± 3.7	0.845	Never intervene (no penalty)
Greedy oracle (1-step)	281.0 ± 20.9	0.625	Myopic expected-reward maximisation
Random	100.1 ± 11.7	0.222	Uniform action selection
Fixed movement_suggestion	35.2 ± 4.8	0.078	Always suggest movement
Fixed journal	50.5 ± 5.1	0.112	Always journal
Fixed goal_reminder	91.2 ± 7.1	0.203	Always remind of goal

Table 1: Upper and lower bounds for the Sprint 1 bootstrap MDP. The exact DP bound (379.1) confirms that always-idle is nearly optimal. The greedy oracle’s myopic one-step lookahead fails badly.

The DP bound (379.1) is computed by backward induction over 450 timesteps on the full state space: `step_bin` (3) $\times$ `sleep` (2) $\times$ action history (64) = 384 states, where action history tracks the last 3 actions for the burden rolling window. The optimal policy selects idle on 99.8% of steps – the DP confirms that never intervening is the optimal strategy for this MDP.

The simulated idle bound (380.4) exceeds the exact DP bound (379.1) by a small margin; this is Monte Carlo noise from 10 seeds (± 3.7 standard error). The exact expected value of always-idle equals the DP bound within numerical precision.

The greedy oracle (281.0) performs 26% below the optimal bound. Myopic one-step lookahead fails because the reward function evaluates the *post-transition* state: an intervention incurs a penalty but also changes `step_bin` and burden, which the greedy oracle correctly prices but cannot amortize over future steps. The long-term value of leaving burden low by not intervening dominates.

3 Base Benchmark Results

50 seeds, 450 timesteps, default hyperparameters (EG $\epsilon = 0.1$, UCB $c = 2.0$, TS $\alpha_0 = \beta_0 = 1$). Config: `configs/sprint1_bootstrap.yaml`.

Agent	Total Reward	Per Step	Last 50 Steps
Epsilon-Greedy ($\epsilon = 0.1$)	280.8 ± 89.3	0.624	0.743
Thompson Sampling ($\alpha_0 = \beta_0 = 1$)	252.6 ± 105.6	0.561	0.662
UCB ($c = 2.0$)	240.4 ± 33.9	0.534	0.803
Random	99.7 ± 10.9	0.222	0.215

Table 2: Base benchmark results (50 seeds, 450 timesteps). Learning agents massively outperform random ($2.5\text{--}2.8\times$), confirming the bootstrap tables contain exploitable structure. EG leads.

Key observation: under bootstrap transitions, learning agents achieve $2.5\text{--}2.8\times$ random, starkly different from the random-transition baseline where all agents clustered within 3% of each other. The LLM-estimated dynamics have meaningful structure.

4 Full Benchmark Results

50 seeds, 450 timesteps, 9 agents with MVP-optimised hyperparameters (EG $\epsilon = 0.05$, D-EG $\epsilon_{\text{start}} = 0.2$, $T_{\text{decay}} = 200$, UCB $c = 0.5$). Config: `configs/sprint1_bootstrap_extensions.yaml`.

Agent	Total Reward	Per Step	Last 50 Steps
Standard UCB ($c = 0.5$)	358.1 ± 14.0	0.796	0.837
Standard D-EG	287.8 ± 106.9	0.640	0.718
Standard EG ($\epsilon = 0.05$)	282.3 ± 95.0	0.627	0.748
Contextual UCB	268.3 ± 29.6	0.596	0.733
Standard TS	252.6 ± 105.6	0.561	0.662
Contextual D-EG	240.4 ± 93.0	0.534	0.614
Contextual EG	216.9 ± 95.7	0.482	0.583
Contextual TS	155.5 ± 70.8	0.346	0.399
Random	99.7 ± 10.9	0.222	0.215
Fixed idle (upper bound)	380.4	0.845	0.845

Table 3: Full benchmark results (50 seeds). Standard UCB at $c = 0.5$ reaches 94.5% of the optimal DP bound. Contextual agents universally underperform their standard counterparts.

Key findings:

1. **Std UCB at $c = 0.5$ is dominant:** 358.1 total, 94.5% of the optimal DP bound (379.1), with the lowest variance (14.0 vs 70–106 for others). Hyperparameter tuning is critical: the default $c = 2.0$ yielded only 240.4.
2. **Contextual agents are worse:** Every contextual variant underperforms its standard counterpart, the opposite of the MVP result. This suggests `step_bin` as a context feature misleads agents in the bootstrap dynamics – transitions are non-stationary (vary by timestep) and the current `step_bin` does not reliably predict intervention efficacy.

3. **The $c = 0.5$ UCB achieves consistency:** with ± 14.0 std dev vs ± 33.9 at $c = 2.0$, tuning simultaneously improves both mean and variance.

5 Masked Reward Benchmark

Same 9 agents on `configs/sprint1.bootstrap_extensions_masked.yaml` where step 4 (end of day) yields zero reward. Results are 50 seeds.

Agent	Total Reward	Per Step	Last 50 Steps
Standard UCB ($c = 0.5$)	265.9 ± 37.3	0.591	0.681
Standard D-EG	248.3 ± 76.2	0.552	0.616
Standard EG ($\epsilon = 0.05$)	195.8 ± 98.4	0.435	0.518
Contextual UCB	178.4 ± 35.2	0.397	0.501
Standard TS	174.6 ± 70.6	0.388	0.456
Contextual D-EG	169.9 ± 70.0	0.378	0.435
Contextual EG	163.7 ± 73.9	0.364	0.443
Contextual TS	119.0 ± 38.5	0.264	0.289
Random	83.6 ± 8.4	0.186	0.179

Table 4: Masked benchmark results (50 seeds, step 4 zero-reward). Relative ordering is preserved. D-EG is most mask-resilient (86% retention).

6 Comparison to Random-Transition Baseline

Metric	Random Transitions	Bootstrap Transitions
Best agent	Std D-EG 189.1 (0.420)	Std UCB 358.1 (0.796)
Random baseline	183.5 (0.408)	99.7 (0.222)
Best vs random gap	3%	259%
Contextual advantage	None ($\approx 0\%$)	Negative (all worse)
Variance (best agent)	± 82	± 14

Table 5: Bootstrap transitions vs random transitions. The LLM-estimated tables produce structured dynamics with large agent differentials, while random transitions wash out all policy differences. Std UCB’s low variance under bootstrap suggests reliable convergence.

7 Context Feature Comparison

The extensions benchmark used `step_bin` as the context feature (3 bins, same fragmentation as `step_bin`). To test whether a better-chosen context could close the gap, we swapped to `burden` (3 levels, same fragmentation) and re-ran the 9-agent benchmark. Results are 50 seeds.

Despite having $13\times$ higher ANOVA F-statistic as a predictor of next-step activity, `burden` makes a worse context feature. The reason is that `burden` is *endogenous* – it is a rolling window count

Agent	No Context	Ctx <code>step_bin</code>	Ctx <code>burden</code>
Standard UCB ($c = 0.5$)	358.1 ± 14.0	—	—
Contextual UCB	—	268.3 ± 29.6	250.6 ± 39.7
Contextual D-EG	—	240.4 ± 93.0	216.0 ± 77.6
Contextual EG	—	216.9 ± 95.7	200.4 ± 82.9
Contextual TS	—	155.5 ± 70.8	158.3 ± 56.1
Random	99.7 ± 10.9	—	—

Table 6: Comparison of context features (50 seeds). Burden context underperforms `step_bin` context across all agents except TS (where the difference is negligible).

of the agent’s past non-idle actions. When a fragmented contextual agent explores, it takes non-idle actions, which increase burden, which shifts the context distribution, which triggers further exploration. This creates a feedback loop:

exploration $\rightarrow$ higher burden $\rightarrow$ unfamiliar context $\rightarrow$ more exploration $\rightarrow \dots$

In contrast, `step_bin` is at least partly exogenous – it depends on the LLM-estimated transition dynamics and is not directly controlled by recent actions. The `step_bin` distribution is relatively stable regardless of the agent’s policy, making it a safer (if less predictive) context feature.

Both context features are dominated by the standard (non-contextual) UCB, confirming that `step_bin` context does not help this MDP.

8 Horizon Scaling

To test whether contextual agents would catch the standard UCB given more data, we re-ran both context-feature benchmarks at 900 days (4500 steps, $10\times$) and 9000 days (45000 steps, $100\times$). All other settings are identical (50 seeds, same hyperparameters).

Several patterns emerge:

1. **Ctx UCB (`step_bin`) effectively closes the gap.** At 90d / 450 steps it achieves 75% of Std UCB’s total; at 900d / 4500 steps it reaches 95%; at 9000d / 45000 steps it reaches 99.3% (37971.9 vs 38241.5). The contextual fragmentation penalty is purely a *sample-efficiency cost*, not an asymptotic limitation. Given enough data, `step_bin`-contextual UCB converges to the same optimal policy of doing nothing.
2. **Ctx UCB (`burden`) also converges, but slower.** Even at 45000 steps, burden-context UCB reaches only 0.8090/step vs Std UCB’s 0.8498 (95.2%). The endogenous feedback loop does not disappear with more data, but its relative penalty shrinks: 75% $\rightarrow$ 89% $\rightarrow$ 95% as horizon increases.
3. **The $10\times$ multiplier gap disappears at longer horizons.** At 900d, contextual agents had higher gain multipliers ($13\text{--}16\times$) than standard ones ($11\text{--}13\times$). At 9000d the per-step rates show a different pattern: Std UCB converges to 0.8498, with most standard agents clustering at 0.823–0.826 and `step_bin`-contextual agents at 0.818–0.844. The gap is now driven by *asymptotic convergence rate* rather than a fixed fragmentation penalty.

Agent	90d / 450 st.	900d / 4500 st.	9000d / 45000 st.	Per-step
Std UCB ($c = 0.5$)	358.1 ± 14.0	3795.7 ± 24.0	38241.5 ± 51.1	0.8498
Ctx UCB (<code>step_bin</code>)	268.3 ± 29.6	3606.6 ± 43.5	37971.9 ± 63.9	0.8438
Ctx UCB (<code>burden</code>)	250.6 ± 39.7	3389.9 ± 79.5	36404.1 ± 374.9	0.8090
Std EG ($\epsilon = 0.05$)	282.3 ± 95.0	3578.6 ± 264.7	37182.9 ± 264.0	0.8263
Ctx EG (<code>step_bin</code>)	216.9 ± 95.7	3376.6 ± 423.0	36824.1 ± 952.6	0.8183
Std D-EG	287.8 ± 106.9	3373.3 ± 916.3	37097.9 ± 2987.5	0.8244
Ctx D-EG (<code>step_bin</code>)	240.4 ± 93.0	3168.0 ± 763.3	37057.1 ± 1590.5	0.8235
Std TS	252.6 ± 105.6	3354.3 ± 842.0	37060.1 ± 3511.0	0.8236
Ctx EG (<code>burden</code>)	200.4 ± 82.9	3140.1 ± 386.4	35008.6 ± 1722.7	0.7780
Ctx D-EG (<code>burden</code>)	216.0 ± 77.6	2694.5 ± 927.8	31957.4 ± 6113.1	0.7102
Ctx TS (<code>step_bin</code>)	155.5 ± 70.8	2215.5 ± 737.6	25537.2 ± 8989.0	0.5675
Ctx TS (<code>burden</code>)	158.3 ± 56.1	1776.2 ± 571.3	19794.5 ± 5612.9	0.4399
Random	99.7 ± 10.9	1020.8 ± 45.1	10177.4 ± 118.4	0.2262

Table 7: Horizon scaling: 90d, 900d, and 9000d (50 seeds). At 9000d Ctx UCB (`step_bin`) reaches 99.3% of Std UCB. Rank ordering is stable across all three horizons.

- Rank ordering is remarkably stable across all three horizons.** No agent that was worse at 450 steps overtakes one that was better at 45000 steps. The relative hierarchy is fully determined early in training. However, some pairs converge: Ctx D-EG (`step_bin`) nearly ties Std TS and Std D-EG at 45000 steps, suggesting the exploration decay eventually lets it converge.
- Std UCB at 45000 steps reaches 0.8498/step, matching the always-idle upper bound (0.845).** The standard deviation shrinks to ± 51.1 , the lowest of any agent at any horizon. UCB with $c = 0.5$ has converged to the optimal deterministic policy of always choosing idle.
- Contextual TS remains terrible even at 45000 steps.** Ctx TS (`step_bin`) at 0.5675/step and Ctx TS (`burden`) at 0.4399/step are the only agents that do not converge toward the standard-agent cluster. Beta-Bernoulli Thompson Sampling with independent per-context Beta posteriors cannot aggregate information across contexts, so it never escapes the fragmentation penalty.
- step_bin beats burden at all three horizons.** The gap between step_bin and burden UCB narrows from 17.7 (90d) to 216.7 (900d) to 1567.8 (9000d), but in relative terms burden per-step stays at 95–96% of step_bin per-step at all three horizons. The relative penalty is constant.

9 Key Findings

- Bootstrap transitions have exploitable structure.** Unlike the Dirichlet-random baselines where all agents performed similarly (~ 185 total), the LLM-estimated tables yield a 259% gap between best learner and random. The LLM produces non-trivial transition dynamics.

2. **Always-idle is the optimal policy.** Backward induction over 384 states confirms the optimal strategy is to never intervene on 99.8% of steps. The optimal DP bound is 379.1 (0.842/step), and Std UCB ($c = 0.5$) reaches 94.5% of this theoretical maximum. The exact DP value matches the always-idle simulation (380.4 ± 3.7) within Monte Carlo noise.
3. **Context features hurt, even with better signal.** Swapping from `step_bin` to `burden` – a context with $13\times$ stronger predictive power – made performance worse, not better. The reason is that `burden` is endogenous: exploration changes `burden`, which shifts the context distribution, which triggers more exploration. This feedback loop amplifies the sample-efficiency cost of context fragmentation.
4. **UCB hyperparameter sensitivity is extreme.** Default $c = 2.0$: 240. Tuned $c = 0.5$: 358. The MVP-optimised values transferred well, but a dedicated grid search may find even better settings.
5. **The greedy oracle (one-step lookahead) is not a good bound** at 281 (26% below the DP optimum). Myopic expected-reward maximisation fails because the reward is computed on the post-transition state: the penalty is immediate but the benefit of intervention (higher `step_bin`) is only realisable in future rewards, which the greedy oracle ignores. Critically, interventions increase `burden`, which degrades future transition probabilities – a negative long-term effect the greedy oracle cannot amortise.
6. **The LLM’s transition estimates are a plausible starting point** but should be validated against real data. The strong idle preference may reflect LLM bias toward inaction rather than true user dynamics.